

# P2650R0

## 2022-11 Library Evolution Polls

Published Proposal, 2022-11-17

**Author:**

[Bryce Adelstein Lelbach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

**Source:**

[GitHub](#)

**Issue Tracking:**

[GitHub](#)

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Audience:**

WG21

---

## Table of Contents

### 1 Introduction

### 2 Voting

#### 2.1 How to Vote

#### 2.2 Voting Timeline

### 3 Poll Structure

#### 3.1 C++23 Polls

##### 3.1.1 [P2164R8] `views::enumerate`

##### 3.1.2 [P2711R1] Making Multi-Param Constructors Of Views Explicit

##### 3.1.3 [P2655R1] `common_reference_t` Of `reference_wrapper` Should Be A Reference Type

##### 3.1.4 FR-015-026 Remove `cbegin/cend` From Spans And Views

##### 3.1.5 FR-013-020 Replace `repeat` With `cycle`

##### 3.1.6 [P2693R0] Formatting `thread::id` And `stacktrace`

- 3.1.7 [LWG3806] Should `Concept formattable<T, charT>` Default To `char`?
- 3.1.8 FR-003-010 Formatting Of Ranges Of tuple-like Objects
- 3.1.9 [P2588R2] Relax `barrier` Phase Completion Step Guarantees
- 3.1.10 US-43-104 `subspan` Type Confusion
- 3.1.11 FR-002-015 Return A span From `strides`
- 3.1.12 GB-087 `start_lifetime_as` Is Broken And Inconsistent For Arrays
- 3.1.13 [P2679R1] Fixing `start_lifetime_as` For Arrays
- 3.1.14 [P2674R1] `is_implicit_lifetime`
- 3.1.15 US-30-072 Predefined Macros From The C Library
- 3.1.16 GB-080 Sync `intmax_t` And `uintmax_t` With C2x
- 3.1.17 [P2652R0] Disallow User Specialization Of `allocator_traits`
- 3.1.18 [P2614R1] Deprecate `numeric_limits::has_denorm`
- 3.1.19 GB-081 Deprecate Contents Of `<stdalign.h>`
- 3.1.20 GB-082 Deprecate Contents Of `<stdbool.h>`
- 3.1.21 GB-084 Deprecate `errc` Constants Related To UNIX STREAMS
- 3.1.22 GB-121 Remove `voidify`
- 3.2 C++26 and Technical Specification Polls
  - 3.2.1 [P2559R1] Plan For Concurrency Technical Specification Version 2
  - 3.2.2 [P2545R1] Why RCU Should Be In C++26
  - 3.2.3 [P2530R1] Why Hazard Pointers Should Be In C++26
  - 3.2.4 [P0493R3] Atomic Maximum/Minimum
  - 3.2.5 [P2527R1] `variant_alternative_index` And `variant_alternative_index_v`
  - 3.2.6 [P2264R4] Make `assert` Macro User Friendly For C And C++

## References

Informative References

## § 1. Introduction

This paper contains the decision polls that the C++ Library Evolution group will take in 2022-11.

## § 2. Voting

These decision polls will be conducted electronically; see [\[P2195R2\]](#) for more information on electronic voting procedures.

All committee members may vote, but if you have not participated in the Library Evolution discussions of a poll, please choose to not vote.

### § 2.1. How to Vote

**[Vote here \(C++23 polls\).](#)**

**[Vote here \(C++26 and Technical Specification polls\).](#)**

A documents.isocpp.org account is necessary to vote. If you have an isocpp.org/papers account, a documents.isocpp.org account has been created for you. Just go to documents.isocpp.org and reset your password. Email [Bryce Adelstein Lelbach](#) if you do not have an account or are having trouble accessing your account.

If you see "Form not found" when you go to the above link, you are not signed in to documents.isocpp.org. Go to documents.isocpp.org and sign in first.

You will not receive a confirmation email after voting. The Library Evolution chairs will send a confirmation to all voters before the close of the poll.

If you need to change how you voted or have any questions or issues about voting, please email [Bryce Adelstein Lelbach](#).

### § 2.2. Voting Timeline

- 2022-11-11: Draft polls distributed by mailing list.
- 2022-11-18: Start of the polling period.
- 2022-12-13: End of the polling period.
- 2022-12-16: Chairs share results and summarize outcomes.

## § 3. Poll Structure

Each poll consists of a statement; you vote on whether you support the statement. All polls will be 5-way polls; you'll vote either "strongly favor", "weakly favor", "neutral", "weakly against", or "strongly against".

Polls phrased as "Return X" indicate that the paper was sent to Library Working Group in the past, and Library Working Group returned it to us with questions. Polls phrased as "Send X" indicate that the paper has not previously been sent to Library Working Group.

All committee members may vote, but if you have not participated in the Library Evolution discussions of a poll (in-person, over telecon, or over email), please consider whether you sufficiently understand the topic to be able to vote in an informed manner. If you do not, please choose to not participate in the poll.

If you do not want to participate in a poll, select the "I do not want to participate in this poll" option. Those who do not participate in a poll will not be counted or recorded. Voting neutral is not the same as choosing to not participate; please don't vote neutral if you did not participate in the discussions of the poll.

Each poll has a comment field. Please write a few sentences explaining the rationale for your vote. We recommend drafting your comments in a text editor and then pasting them into the polling system.

Poll results and selected comments will be shared with Library Evolution after the end of the polling period, including attribution of your votes and comments.

If you want to discuss the polls or how you plan on voting, please start an email thread on the [Library Evolution mailing list](#).

You will not receive a confirmation email after voting.

If you need to see how you voted, change how you voted, or have any questions or issues about voting, please contact the Library Evolution chair.

### § 3.1. C++23 Polls

#### § 3.1.1. [\[P2164R8\]](#) `views::enumerate`

Send [\[P2164R8\]](#) `views::enumerate` to Library Working Group for C++23, classified as an addition ([\[P0592R4\]](#) bucket 3 item).

### § 3.1.2. [\[P2711R1\]](#) Making Multi-Param Constructors Of Views Explicit

Send [\[P2711R1\]](#) Making Multi-Param Constructors Of Views Explicit to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.3. [\[P2655R1\]](#) `common_reference_t` Of `reference_wrapper` Should Be A Reference Type

Send [\[P2655R1\]](#) `common_reference_t` Of `reference_wrapper` Should Be A Reference Type to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.4. [FR-015-026](#) Remove `cbegin/cend` From Spans And Views

Reject C++23 National Body comment [FR-015-026](#) Remove `cbegin/cend` From Spans And Views and any duplicate comments.

### § 3.1.5. [FR-013-020](#) Replace `repeat` With `cycle`

Reject C++23 National Body comment [FR-013-020](#) Replace `repeat` With `cycle` and any duplicate comments.

### § 3.1.6. [\[P2693R0\]](#) Formatting `thread::id` And `stacktrace`

Send [\[P2693R0\]](#) Formatting `thread::id` And `stacktrace` to Library Working Group for C++23, classified as an addition ([\[P0592R4\]](#) bucket 3 item).

### § 3.1.7. [\[LWG3806\]](#) Should Concept `formattable<T, charT>` Default To `char`?

Reject [\[LWG3806\]](#) Should Concept `formattable<T, charT>` Default To `char`? and any duplicate issues.

### § 3.1.8. [FR-003-010](#) Formatting Of Ranges Of tuple-like Objects

Reject C++23 National Body comment [FR-003-010](#) Formatting Of Ranges Of tuple-like Objects and any duplicate comments.

### § 3.1.9. [\[P2588R2\]](#) Relax barrier Phase Completion Step Guarantees

Send [\[P2588R2\]](#) Relax barrier Phase Completion Step Guarantees to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.10. [US-43-104](#) subspan Type Confusion

Reject C++23 National Body comment [US-43-104](#) subspan Type Confusion and any duplicate comments.

### § 3.1.11. [FR-002-015](#) Return A span From strides

Reject C++23 National Body comment [FR-002-015](#) Return A span From strides and any duplicate comments.

### § 3.1.12. [GB-087](#) start\_lifetime\_as Is Broken And Inconsistent For Arrays

Reject C++23 National Body comment [GB-087](#) start\_lifetime\_as Is Broken And Inconsistent For Arrays as per [\[P2679R1\]](#) Fixing start\_lifetime\_as For Arrays .

### § 3.1.13. [\[P2679R1\]](#) Fixing start\_lifetime\_as For Arrays

Send [\[P2679R1\]](#) Fixing start\_lifetime\_as For Arrays to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.14. [\[P2674R1\]](#) is\_implicit\_lifetime

Send [\[P2674R1\]](#) is\_implicit\_lifetime to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.15. [US-30-072](#) Predefined Macros From The C Library

Reject C++23 National Body comment [US-30-072](#) Predefined Macros From The C Library and any duplicate comments.

### § 3.1.16. [GB-080](#) Sync `intmax_t` And `uintmax_t` With C2x

Send the proposed resolution to C++23 National Body comment [GB-080](#) Sync `intmax_t` And `uintmax_t` With C2x to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.17. [\[P2652R0\]](#) Disallow User Specialization Of `allocator_traits`

Send [\[P2652R0\]](#) Disallow User Specialization Of `allocator_traits` to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.18. [\[P2614R1\]](#) Deprecate `numeric_limits::has_denorm`

Send [\[P2614R1\]](#) Deprecate `numeric_limits::has_denorm` to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.19. [GB-081](#) Deprecate Contents Of `<stdalign.h>`

Send the proposed resolution to C++23 National Body comment [GB-081](#) Deprecate Contents Of `<stdalign.h>` to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.20. [GB-082](#) Deprecate Contents Of `<stdbool.h>`

Send the proposed resolution to C++23 National Body comment [GB-082](#) Deprecate Contents Of `<stdbool.h>` to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.21. [GB-084](#) Deprecate `errc` Constants Related To UNIX STREAMS

Send the proposed resolution to C++23 National Body comment [GB-084](#) Deprecate `errc` Constants Related To UNIX STREAMS to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

### § 3.1.22. [GB-121](#) Remove voidify

Send the proposed resolution to C++23 National Body comment [GB-121](#) Remove voidify to Library Working Group for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

## § 3.2. C++26 and Technical Specification Polls

### § 3.2.1. [\[P2559R1\]](#) Plan For Concurrency Technical Specification Version 2

Send [\[P2559R1\]](#) Plan For Concurrency Technical Specification Version 2 to Library Working Group for the Concurrency Technical Specification Version 2.

### § 3.2.2. [\[P2545R1\]](#) Why RCU Should Be In C++26

Send [\[P2545R1\]](#) Why RCU Should Be In C++26 to Library Working Group for C++26, classified as an addition ([\[P0592R4\]](#) bucket 3 item).

### § 3.2.3. [\[P2530R1\]](#) Why Hazard Pointers Should Be In C++26

Send [\[P2530R1\]](#) Why Hazard Pointers Should Be In C++26 to Library Working Group for C++26, classified as an addition ([\[P0592R4\]](#) bucket 3 item).

### § 3.2.4. [\[P0493R3\]](#) Atomic Maximum/Minimum

Send [\[P0493R3\]](#) Atomic Maximum/Minimum to Library Working Group for C++26, classified as an addition ([\[P0592R4\]](#) bucket 3 item).

### § 3.2.5. [\[P2527R1\]](#) `variant_alternative_index` And `variant_alternative_index_v`

Send [\[P2527R1\]](#) `variant_alternative_index` And `variant_alternative_index_v` to Library Working Group for C++26, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).



### § 3.2.6. [P2264R4] Make `assert` Macro User Friendly For C And C++

Send [P2264R4] Make `assert` Macro User Friendly For C And C++ to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).

## § References

### § Informative References

#### [LWG3806]

Jonathan Wakely. *Should concept formattable<T, charT> default to char?*. New. URL: <https://wg21.link/lwg3806>

#### [P0493R3]

Al Grant, Bronek Kozicki, Tim Northover. *Atomic maximum/minimum*. 18 December 2021. URL: <https://wg21.link/p0493r3>

#### [P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

#### [P2164R8]

Corentin Jabot. *views::enumerate*. 2022-11-11. URL: <https://wg21.link/P2164R8>

#### [P2195R2]

Bryce Adelstein LeBach. *Electronic Straw Polls*. 28 January 2021. URL: <https://wg21.link/p2195r2>

#### [P2264R4]

Peter Sommerlad. *Make assert() macro user friendly for C and C++*. 8 August 2022. URL: <https://wg21.link/p2264r4>

#### [P2527R1]

Alex Christensen. *std::variant alternative index and std::tuple element index*. 20220930. URL: <https://wg21.link/p2527r1>

#### [P2530R1]

Maged Michael, Maged M. Michael, Michael Wong, Paul McKenney, Andrew Hunter, Daisy S. Hollman, JF Bastien, Hans Boehm, David Goldblatt, Frank Birbacher, Mathias Stearn. *Why Hazard Pointers should be in C++26*. 12 October 2022. URL: <https://wg21.link/p2530r1>

#### [P2545R1]

Paul E. McKenney, Michael Wong, Maged M. Michael, Geoffrey Romer, Andrew Hunter, Arthur O'Dwyer, Daisy Hollman, JF Bastien, Hans Boehm, David Goldblatt, Frank Birbacher, Erik

Rigtorp, Tomasz Kamiński, Jens Maurer. *Why RCU Should be in C++26*. 7 October 2022. URL: <https://wg21.link/p2545r1>

**[P2559R1]**

Bryce Adelstein Leibach. *Plan for Concurrency Technical Specification Version 2*. 2022-11-17. URL: <https://wg21.link/P2559R1>

**[P2588R2]**

Gonzalo Brito Gadeschi; et al. *barrier's completion phase guarantees*. 2022-11-07. URL: <https://wg21.link/P2588R2>

**[P2614R1]**

Matthias Kretz. *Deprecate numeric\_limits::has\_denorm*. 27 September 2022. URL: <https://wg21.link/p2614r1>

**[P2652R0]**

Pablo Halpern. *Disallow user specialization of allocator\_traits*. 10 October 2022. URL: <https://wg21.link/p2652r0>

**[P2655R1]**

Hui Xie; S. Levent Yilmaz. *`common\_reference\_t` of `reference\_wrapper` Should Be a Reference Type*. 2022-10-17. URL: <https://wg21.link/P2655R1>

**[P2674R1]**

Timur Doumler; Vittorio Romeo. *A trait for implicit lifetime types*. 2022-10-14. URL: <https://wg21.link/P2674R1>

**[P2679R1]**

Timur Doumler; et al. *Fixing std::start\_lifetime\_as and std::start\_lifetime\_as\_array*. 2022-11-11. URL: <https://wg21.link/P2679R1>

**[P2693R0]**

Corentin Jabot; Victor Zverovich. *Formatting thread::id and stacktrace*. 2022-11-11. URL: <https://wg21.link/P2693R0>

**[P2711R1]**

Ville Voutilainen. *Making multi-param constructors of views explicit*. 2022-11-12. URL: <https://wg21.link/P2711R1>